Projekt z kryptografii

Skład zespołu: Aleksandra Łuszczyk (Liderka), Aleksandra Jankowska, Adam Mazanowski

Wpływ jakości generatorów liczb pseudolosowych na bezpieczeństwo kluczy RSA

Koncepcja realizacji projektu, podział zadań i harmonogram

1. Opis i cel projektu

Bezpieczeństwo RSA opiera się na tym, że trudno rozłożyć moduł $N = p \cdot q$ na czynniki pierwsze. Zakłada się przy tym, że liczby pierwsze p i q są losowe, tajne i różne dla każdego klucza. Same klucze są więc tak dobre, jak generator liczb losowych, z którego powstały. Jeśli generator jest słaby, to klucz może być złamany, mimo że sam algorytm RSA jest poprawny i długość klucza jest duża.

Błędy generatora mogą prowadzić do kilku różnych problemów:

- **mała przestrzeń ziaren:** atakujący może przejrzeć wszystkie możliwe ziarna, wygenerować te same klucze i znaleźć ten, który pasuje do modułu N (tak było w znanym błędzie OpenSSL w Debianie w 2008 roku, gdzie ziarno zależało praktycznie tylko od numeru procesu);
- **powtarzające się liczby pierwsze:** dla kluczy $N_1 = p \cdot q_1$ i $N_2 = p \cdot q_2$ wystarczy policzyć $NWD(N_1, N_2)$ algorytmem Euklidesa, żeby dostać p i rozłożyć oba klucze;
- **złą strukturę liczb pierwszych:** np. p i q bardzo bliskie siebie, co pozwala je znaleźć metodą Fermata.

Po odzyskaniu p i q liczymy $\varphi(N) = (p - 1)(q - 1)$ oraz klucz prywatny $d = e^{-1} \bmod \varphi(N)$. W 2012 roku badacze (m.in. Lenstra i współautorzy oraz Heninger i współautorzy) przeskanowali klucze publiczne z internetu i znaleźli tysiące kluczy ze wspólnymi czynnikami, więc problem nie jest tylko teoretyczny.

Cel projektu: sprawdzić w praktyce, jak jakość generatora liczb pseudolosowych wpływa na bezpieczeństwo kluczy RSA. Zbudujemy kilka generatorów o różnej jakości (od bardzo słabego do kryptograficznie bezpiecznego), wygenerujemy z nich klucze RSA, zaatakujemy je różnymi metodami i porównamy skuteczność oraz czas ataków.

2. Zakres projektu

Zakres obejmuje zadania podane w opisie tematu:

1. Definicja zagadnienia: czym jest generator pseudolosowy, czym różni się generator kryptograficzny od zwykłego i jak jego jakość wpływa na RSA.
2. Implementacja prostej wersji (atak NWD na klucze RSA ze słabego generatora) i jej rozszerzenia.
3. Implementacja generatorów liczb losowych w Pythonie, o różnej jakości.
4. Generowanie kluczy RSA z użyciem tych generatorów.
5. Znalezienie i opisanie rodzajów ataków na klucze wygenerowane słabym generatorem.
6. Zbudowanie środowiska testowego.
7. Przeprowadzenie testów.
8. Porównanie skuteczności ataków oraz wpływu jakości generatora na wyniki.

3. Koncepcja realizacji

3.1 Generatory o różnej jakości

Chcemy porównać generatory na kilku poziomach jakości. Wstępnie planujemy cztery:

Nr	Generator	Opis i spodziewany efekt
G1	Bardzo słaby	Ziarno z małego zakresu (np. 16 bitów). Spodziewamy się, że wszystkie klucze da się odtworzyć przez przeszukanie ziaren.
G2	Słaby (mała entropia)	Liczby pierwsze losowane z ograniczonej puli (rozmiar puli jest parametrem, np. 10, 50, 500). Spodziewamy się wspólnych dzielników w zbiorze kluczy.
G3	Średni (niekryptograficzny)	Mersenne Twister (moduł random w Pythonie) z ziarnem zależnym od czasu. Dobry statystycznie, ale nie jest przeznaczony do kryptografii, a ziarno jest przewidywalne.
G4	Dobry (kontrolny)	Generator kryptograficzny (moduł secrets / os.urandom, w porównaniu z OpenSSL). Atak na te klucze nie powinien się udać.

Dla każdego generatora zmierzymy też prostą miarę jakości: entropię Shannona wyniku, test częstości bitów oraz liczbę unikalnych liczb pierwszych w próbie kluczy. Dzięki temu będziemy mogli powiązać „jakość generatora" z odpornością kluczy na ataki.

3.2 Generowanie kluczy RSA

Dla każdego generatora generujemy klucze tym samym algorytmem: wybór dwóch liczb pierwszych (test Millera–Rabina), obliczenie N i $\varphi(N)$, e = 65537 i wyliczenie d. Klucze zapisujemy w formacie PEM, żeby dało się je wczytać w OpenSSL. Przygotujemy zestawy o różnych parametrach: długość klucza 512, 1024 i 2048 bitów oraz liczba kluczy od kilkunastu do kilku tysięcy. Poprawność kluczy sprawdzimy w OpenSSL (np. openssl rsa -check).

3.3 Ataki

- **NWD parami (wersja prosta):** dla każdej pary modułów liczymy NWD. Jeśli wynik jest większy od 1, mamy wspólną liczbę pierwszą i odtwarzamy oba klucze prywatne. Wadą jest złożoność rzędu n^2 par.
- **Batch GCD:** szybsza wersja dla dużych zbiorów kluczy, oparta na drzewie iloczynów.
- **Faktoryzacja Fermata:** skuteczna, gdy p i q są bardzo bliskie siebie.
- **Przeszukanie przestrzeni ziaren:** zakładamy, że znamy algorytm generacji (jak w zasadzie Kerckhoffsa) i sprawdzamy kolejne ziarna, aż wygenerowany moduł zgodzi się z N. Działa tylko, gdy przestrzeń ziaren jest mała.

Odtworzony klucz prywatny sprawdzamy w praktyce: odszyfrowanie wiadomości zaszyfrowanej kluczem publicznym i weryfikacja w OpenSSL.

3.4 Środowisko testowe

Pracujemy we wspólnym repozytorium (Git). Środowisko: Python 3 z wirtualnym środowiskiem (venv) i plikiem requirements.txt, OpenSSL oraz Jupyter Notebook. Projekt będzie dostępny jako skrypt w wierszu poleceń (podkomendy do generowania kluczy, ataku i pomiarów) oraz jako notebook z opisem, eksperymentami i wykresami. Dzięki temu każdy z zespołu może powtórzyć wyniki u siebie.

Proponowana struktura repozytorium:

Element	Zawartość
generators.py	generatory G1–G4
quality.py	miary jakości generatorów (entropia, test częstości bitów, liczba unikalnych p)
keygen.py	generowanie kluczy RSA, zapis do PEM
attacks.py	NWD parami, batch GCD, Fermat, przeszukanie ziaren, odtwarzanie klucza prywatnego
benchmark.py	pomiary czasu i skuteczności
cli.py	skrypt w wierszu poleceń (generate / attack / benchmark)
notebook.ipynb	eksperymenty, wykresy, wnioski
tests/	testy poprawności (w tym sprawdzenie z OpenSSL)
README.md	instrukcja uruchomienia krok po kroku

3.5 Testy i porównanie

Każdy atak uruchomimy na zestawach kluczy z każdego generatora. Zmierzymy skuteczność (jaki procent kluczy udało się złamać) i czas działania, a także zależność od liczby kluczy, długości klucza, rozmiaru puli i przestrzeni ziaren. Pomiary powtórzymy kilka razy i podamy średnią. Główny wynik to tabela „generator × atak" (który atak działa na który generator) oraz wykresy, np. czas ataku w zależności od liczby kluczy dla NWD parami i batch GCD. Na wykresach i w tabeli pokażemy, jak ze wzrostem jakości generatora skuteczność ataków spada do zera. Ponadto uwzględniamy ryzyko związane z potencjalnym wąskim gardłem obliczeniowym przy testowaniu bardzo dużych zbiorów kluczy. Zakładamy, że w przypadku, gdyby obciążenie było zbyt duże i czas wykonywania natywnych operacji matematycznych w Pythonie (szczególnie przy analizie Batch GCD) okazał się zbyt długi, płynnie przejdziemy na zoptymalizowane biblioteki zewnętrzne takie jak gmpy2, co pozwoli zachować terminowość prowadzonych eksperymentów.

4. Podział zadań w zespole

Zespół ma trzy osoby. Każda odpowiada za jeden obszar, ale na bieżąco pomagamy sobie nawzajem i wspólnie robimy przegląd kodu.

Osoba	Zadania	Rezultat
Aleksandra Łuszczyk Generatory i klucze	• Opis zagadnienia i przegląd literatury (PRNG a CSPRNG, entropia, rola w RSA). • Implementacja generatorów G1–G4. • Generowanie kluczy RSA, zapis do PEM, weryfikacja w OpenSSL. • Miary jakości generatorów i zestawy kluczy do testów. • Część teoretyczna raportu.	generators.py, quality.py, keygen.py, zestawy kluczy, rozdział teoretyczny

Osoba	Zadania	Rezultat
Aleksandra Jankowska Ataki	• Znalezienie i opis rodzajów ataków na słabo wygenerowane klucze RSA. • Implementacja ataku NWD i odtwarzania klucza prywatnego. • Rozszerzenia: batch GCD, faktoryzacja Fermata, przeszukanie ziaren. • Sprawdzenie, że odzyskany klucz działa (odszyfrowanie, OpenSSL). • Część raportu o atakach.	attacks.py, opis ataków
Adam Mazanowski Środowisko, testy, analiza	• Repozytorium, środowisko (venv, OpenSSL, Jupyter), README. • Skrypt w wierszu poleceń (CLI) łączący generowanie i ataki. • Testy poprawności i skrypt do pomiarów (czas, skuteczność). • Tabela „generator × atak”, wykresy, wnioski i zalecenia. • Część raportu z wynikami.	cli.py, benchmark.py, tests/, notebook, wykresy

Wspólnie: ustalenie parametrów testów, przegląd kodu (każdy sprawdza część innej osoby), złożenie raportu i przygotowanie prezentacji.

5. Harmonogram

Etapy	Aleksandra Łuszczyk	Aleksandra Jankowska	Adam Mazanowski	Kamień milowy
Etap 1 Start (21 października)	Opis zagadnienia, przegląd literatury, projekt generatorów G1–G4.	Przegląd rodzajów ataków, wybór do realizacji, przykład NWD na małych liczbach.	Repozytorium, środowisko (venv, OpenSSL, Jupyter), szkielet CLI.	Temat zdefiniowany, środowisko działa u wszystkich.
Etap 2 Dyskusja koncepcji projektu (18 listopada)	Implementacja generatorów i generowania kluczy (PEM).	Atak NWD parami i odtwarzanie klucza prywatnego.	Testy poprawności, weryfikacja kluczy w OpenSSL, szkielet notebooka.	Prosta wersja działa od początku do końca.
Etap 3 Prezentacje w grupach na żywo (16 grudnia)	Miary jakości generatorów, zestawy kluczy o różnych parametrach.	Batch GCD, faktoryzacja Fermata, przeszukanie ziaren.	Skrypt pomiarowy, zbieranie statystyk czasu i skuteczności.	Wszystkie ataki działają, dane zebrane.
Etap 4 Przedstawienie wyników finalnych (27 stycznia)	Rozdział teoretyczny raportu, zalecenia jak poprawnie generować klucze.	Rozdział o atakach, uporządkowanie kodu.	Tabela generator × atak, wykresy, finalizacja notebooka i CLI.	Raport i prezentacja gotowe, oddanie projektu.

6. Rezultaty końcowe

- Kod w Pythonie w repozytorium: generatory, generowanie kluczy, ataki, miary jakości i skrypt w wierszu poleceń.
- Jupyter Notebook z eksperymentami, wykresami i wnioskami.
- Zestawy testowych kluczy RSA oraz wyniki pomiarów.
- Raport z opisem zagadnienia, rodzajów ataków i porównaniem wpływu jakości generatora na bezpieczeństwo kluczy.
- Krótka prezentacja projektu.